Program 7

Title: Working with vector database-3(Building search system).

Problem statement:

Write the python program to build the similarity search system which search the query from a document by creating various index and evaluate the performance of search for all index created by setting benchmark of parameters.

Method:

1. Import FAISS library
2. Get the vectors using any of the method mentioned above
3. Create all kind of index and save and load index.
4. Set the benchmark (preferably large dimension) comparing HNSW index.
5. Get the execution time.

Steps

Use Windows Command Prompt for the steps below. Internet is needed to install Python and the packages.

1. Check the Python version Press Windows + R, type cmd and press Enter. Run the command below. If a Python 3 version appears, continue to Step 3.

```
python --version
```

2. Install Python if needed If no version appears, try py --version.

3. Create and open the Desktop folder On the Desktop, Name it Program7. Open this folder, click the address bar, type cmd and press Enter. Command Prompt now opens inside Program7. Run the following commands one at a time in this window.

4. Create an environment Run the commands below in the same window. The prompt should begin with (.venv).

```
python -m venv .venv  (or) py -m venv .venv

.venv\Scripts\activate
```

5. Install libraries Wait until installation completes.

```
python -m pip install --upgrade pip

python -m pip install numpy faiss-cpu scikit-learn
```

6. Check installation The command should print Installation successful.

```
python -c "import faiss, numpy, sklearn; print('Installation successful')"
```

7. Save the code: Open Notepad++, type the following code, and save as **program7.py** in the **Program7 folder**.

```
import time                    # Measure execution time.
import faiss                   # Create and search vector indexes.
import numpy as np             # Numerical array library.
from sklearn.feature_extraction.text import HashingVectorizer

# Sample document paragraphs.
documents = [
    'FAISS performs similarity search on vectors.',
    'Flat index checks all vectors for exact search.',
    'IVF divides vectors into clusters for searching.',
    'HNSW uses a graph for fast similarity search.',
    'Python is used to develop software applications.',
    'Databases store and retrieve information.',
    'Machine learning learns patterns from data.',
    'An index can be saved and loaded from a file.'
]

query = ['graph similarity search']  # Text to search.
DIM, K = 512, 3                # Vector size and number of results.

# Convert words into normalized numerical features.
vectorizer = HashingVectorizer(
    n_features=DIM, alternate_sign=False, norm='l2'
)

# Encode document and query using the same method.
x = vectorizer.transform(documents).toarray().astype('float32')
q = vectorizer.transform(query).toarray().astype('float32')

faiss.omp_set_num_threads(1)       # Use one CPU thread.

# Flat: check every vector.
flat = faiss.IndexFlatL2(DIM)

# IVF: create two clusters and search one.
ivf = faiss.IndexIVFFlat(faiss.IndexFlatL2(DIM), DIM, 2)
ivf.cp.min_points_per_centroid = 1   # Allow the tiny training example.
ivf.nprobe = 1

# HNSW: connect nearby vectors in a graph.
hnsw = faiss.IndexHNSWFlat(DIM, 16)
hnsw.hnsw.efConstruction = 40       # Exploration during graph building.
hnsw.hnsw.efSearch = 32             # Exploration during searching.

indexes = {'Flat': flat, 'IVF': ivf, 'HNSW': hnsw}

print('Index    Build(ms)   Search(ms)')
```

```
# Build and test each index.
for name, index in indexes.items():
    start = time.perf_counter()

    if not index.is_trained:
        index.train(x)            # Train IVF before adding vectors.

    index.add(x)                  # Store document vectors.
    build_ms = (time.perf_counter() - start) * 1000

    # Save the index and load it again.
    faiss.write_index(index, name + '.index')
    index = faiss.read_index(name + '.index')

    index.search(q, K)            # Warm up before timing.
    start = time.perf_counter()

    # Repeat the same query to measure average search time.
    for _ in range(100):
        distances, ids = index.search(q, K)

    search_ms = (time.perf_counter() - start) * 1000 / 100

    # Display times in milliseconds.
    print(f'{name:8} {build_ms:9.3f} {search_ms:12.3f}')
    print('Matching paragraphs:')

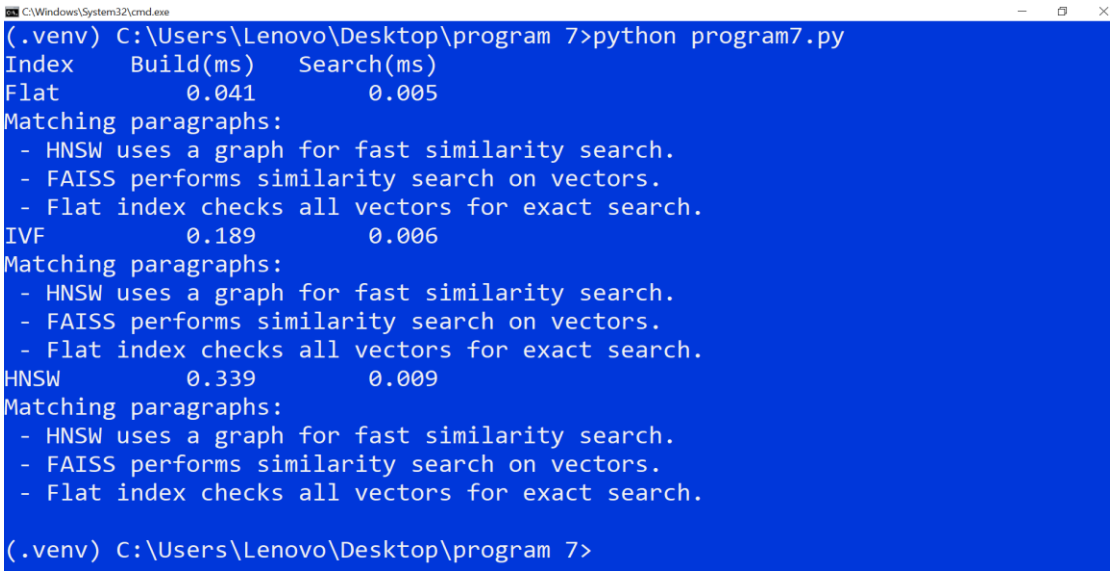
    # Convert returned IDs into paragraph text.
    for i in ids[0]:
        if i >= 0:                # Skip invalid IDs.
            print(' -', documents[i])
```

8. Run the program Return to the same Command Prompt and execute:

python program7.py

9. Check the results The screen shows build time, search time and matching paragraphs for each index. The Program7 folder contains Flat.index, IVF.index and HNSW.index. The program saves and reloads these automatically.

Output



Notes

1. The program searches for **"graph similarity search"** in the eight document paragraphs.
2. **Flat, IVF and HNSW** each return the three closest paragraphs.
3. The same paragraphs appear three times because **each index displays its own results**.
4. **Build time** is the time taken to prepare an index and add the document vectors.
5. **Search time** is the average time taken to search the query, calculated over 100 repetitions.
6. A **lower time means faster execution**. In your output, Flat has the lowest build and search times.
7. Flat is faster here because the dataset contains **only eight paragraphs**.
8. For **large datasets**, HNSW often provides faster approximate search, but its graph takes longer to build. Flat generally builds quickly, while searching requires checking every vector.
9. The fastest index depends on the **dataset, parameter settings and computer**. HNSW is not guaranteed to be fastest in every run.

What we are doing in this program

1. Take **eight document paragraphs** as sample data.
2. Provide a search query: **"graph similarity search."**
3. Convert the paragraphs and query into **numerical vectors**.
4. Create three FAISS indexes: **Flat, IVF and HNSW**.
5. Train the index where required and **add the document vectors**.
6. **Save and reload** each index.
7. Search each index to find the **three closest paragraphs**.
8. Display the matching text and compare **build time and average search time**.

Purpose: To understand how vector similarity search retrieves matching text and how different indexes affect execution time.